Martin Bergstø

Ontology-Driven Decision Support for Machine Learning Method Selection in Product Development and Production

Combining Semantic Literature Retrieval with Ontology-Based Filtering and Implementation Support

Master's thesis in Engineering and ICT
Supervisor: Andrei Lobov
June 2026

Norwegian University of Science and Technology
Faculty of Engineering
Department of Mechanical and Industrial Engineering

NTNU

ABSTRACT

Write an abstract/summary of your thesis, and state your main findings here.

A summary should be included in both English and any second language, if this is applicable, regardless if the thesis is written in English or in your preferred language. These should be on separate pages, the English version first.

PREFACE

Write the preface of your thesis here.

You may include acknowledgements and thanks as part of your preface on this page, or you may add it as a new chapter after the preface.

CONTENTS

LIST OF FIGURES

LIST OF TABLES

ABBREVIATIONS

List of all abbreviations in alphabetic order:

- **AIoDP** AI-on-Demand Platform
- **AutoML** Automated Machine Learning
- **BERT** Bidirectional Encoder Representations from Transformers
- **BM25** Best Match 25
- **HPC** High-Performance Computing
- **IR** Information Retrieval
- **ML** Machine Learning
- **NTNU** Norwegian University of Science and Technology
- **OMA-ML** Ontology-Based Meta AutoML
- **OWL** Web Ontology Language
- **OWL-DL** OWL Description Logic
- **PROV-O** Provenance Ontology
- **RDF** Resource Description Framework
- **RDFS** RDF Schema
- **SBERT** Sentence-BERT
- **SHACL** Shapes Constraint Language
- **SKOS** Simple Knowledge Organization System
- **SME** Small and Medium-sized Enterprise
- **SPARQL** SPARQL Protocol and RDF Query Language
- **SQL** Structured Query Language
- **tf-idf** Term Frequency-Inverse Document Frequency
- **W3C** World Wide Web Consortium

CHAPTER

ONE

INTRODUCTION

This chapter introduces the purpose and overall direction of the thesis. It presents the motivation for the study and describes the main problem that is addressed. The research questions are then defined to guide the work. Finally, the chapter provides an overview of the structure of the thesis.

1.1 Motivation

Machine learning has seen strong growth in recent years and is now widely applied in product development and production. It is used for tasks such as optimization, control, prediction, and decision support (Wuest et al. 2016).

Advances in data availability and computing power have further increased the use of machine learning in industrial settings (Rai et al. 2021). As a result, machine learning plays an important role in improving efficiency, supporting decision-making, and enabling new capabilities in engineering and manufacturing.

At the same time, the field of machine learning is large and diverse. Many different algorithms, methods, and approaches are available, each with its own strengths and weaknesses (Wuest et al. 2016). There is no single method that performs best for all problems, and performance depends on the specific application and data (Nti et al. 2022a; I. Lee and Shin 2020).

This creates a challenge for engineers and practitioners, who must select suitable methods for their problems. The large number of available options can act as a barrier and limit the effective use of machine learning in practice (Nti et al. 2022a). Selecting an appropriate method often requires balancing multiple factors such as accuracy, interpretability, and data requirements (Plathottam et al. 2023a; Ali, S. Lee, and Chung 2017).

If the selection is not done carefully, the resulting model may perform poorly or fail to provide useful insights (Ali, S. Lee, and Chung 2017). In practice, simpler models are often preferred due to their interpretability, even if more complex models may offer higher accuracy (Paleyes, Urma, and Lawrence 2022). This highlights the importance of considering trade-offs in real-world applications.

Another challenge is the gap between research and practical implementation. Research results are distributed across many articles and domains, which makes it difficult to gain a complete overview (Nti et al. 2022a). In addition, most studies focus on developing models, while less attention is given to how they should be selected and applied in practice.

Companies also face challenges related to lack of knowledge and experience when adopting machine learning (Sonntag et al. 2023). In many cases, the selection of methods is based on trial and error, which is time-consuming and inefficient (Sonntag et al. 2023). There is also often pressure to adopt machine learning without a clear plan, which increases the risk of unsuccessful implementations (Plathottam et al. 2023a).

These challenges highlight the need for more structured approaches that can support method selection and make machine learning more accessible.

1.2 Problem Description

Despite the growing adoption of machine learning in manufacturing and engineering, practitioners often lack adequate support when selecting appropriate methods for their specific problems. Although a large number of use cases exist, there is currently no guidance or standard for developing machine learning solutions from ideation to deployment, and it remains difficult for non-experts to begin developing solutions for their specific problems (Tingting Chen et al. 2023). In practice, method selection is often based on trial and error, which is both time-consuming and dependent on prior experience that many practitioners do not have (Sonntag et al. 2023).

A central limitation is that no structured approach currently exists for making an informed selection of machine learning algorithms based on a well-defined problem description (Sonntag et al. 2023). Existing frameworks tend to use accuracy or prediction speed as the primary selection criteria, which is often too general and can lead to unsuitable methods being chosen (Sala et al. 2018). Manual evaluation of candidate algorithms is highly time-consuming, and if the selection is not done carefully, the resulting model may fail to produce useful results (Ali, S. Lee, and Chung 2017).

Existing research further focuses primarily on the development and evaluation of machine learning models, while less attention is given to what conditions lead to selecting one approach over another in specific industrial contexts (Arena et al. 2024). The lack of interpretability in many approaches is also a key barrier to adoption, as practitioners need to understand and trust the recommendations they receive (Presciuttini et al. 2024).

At the same time, the relevant knowledge is distributed across a large and fragmented body of literature. The extensive number of works on machine learning algorithms and applications creates broad knowledge on the topic, but may also generate disorientation when selecting the right approach (Sala et al. 2018). This knowledge is difficult to access and apply systematically during method selection, which limits the ability to make informed, evidence-based decisions (Nti et al. 2022a).

Addressing these challenges requires a structured approach that can translate problem descriptions into method recommendations and connect those recommendations to relevant research. This thesis investigates whether combining ontology-based knowledge representation with semantic retrieval from scientific literature can provide such support, offering interpretable recommendations grounded in prior research alongside guidance for early-stage implementation.

Burde MLguide forklares i så stor detalj? Og bør det forklares her eller under research questions?

1.3 Research Questions

The challenges outlined above point to a need for structured decision support that can connect problem descriptions with suitable machine learning methods, use the research literature to justify those recommendations, and lower the barrier to early-stage implementation. This thesis investigates whether a system combining these capabilities can provide useful and justified recommendations in practice. The following hypothesis is proposed:

> A decision-support system that combines ontology-based knowledge representation, semantic similarity between problem descriptions and scientific literature, and structured implementation support can provide relevant and justified machine learning method recommendations for users with limited machine learning expertise.

To investigate this hypothesis, the following research questions are defined:

- How can ontology-based knowledge improve machine learning method selection?
- How can semantic similarity between problem descriptions and literature support recommendations?
- How can literature-based evidence support and justify machine learning method recommendations?
- To what extent can such a system support early-stage machine learning implementation?

To investigate the hypothesis and research questions, a web-based decision-support system, called MLguide, is developed that combines structured knowledge with information from scientific literature.

1.4 Thesis Structure

The remainder of this thesis is structured as follows:

- Chapter 1 introduces the motivation, problem description, and research questions
- Chapter 2 presents background and related work
- Chapter 3 describes the methodology
- Chapter 4 presents the system implementation
- Chapter 5 presents the results
- Chapter 6 discusses the findings and limitations
- Chapter 7 concludes the thesis and suggests future work

CHAPTER

TWO

BACKGROUND AND STATE OF THE ART

2.1 Machine Learning

2.1.1 Machine Learning Paradigms and Tasks

Machine learning refers to algorithms that improve their performance at some task through experience (Mitchell 1997). Such algorithms can be broadly categorized according to the type of supervision they receive during that experience, also called training. There are three main paradigms commonly distinguished in the literature: supervised learning, unsupervised learning, and reinforcement learning (Bishop 2006) (Goodfellow, Bengio, and Courville 2016, Chapter 5) (Sarker 2021). The following sections outline each paradigm and introduce some common tasks.

In supervised learning, the training data consists of input vectors paired with corresponding target values. The two most common supervised tasks are classification, where the goal is to assign an input to one of a finite number of discrete categories, and regression, where the goal is to predict a continuous numerical value (Bishop 2006) (Goodfellow, Bengio, and Courville 2016, Chapter 5).

In unsupervised learning, no target values are provided. The algorithm must instead identify structure in the data directly (Bishop 2006). Common tasks include clustering, dimensionality reduction, and anomaly detection (Sarker 2021) (Goodfellow, Bengio, and Courville 2016, Chapter 5). Clustering groups data points such that objects within the same group are more similar to each other than to those in other groups (Sarker 2021). Dimensionality reduction simplifies data by reducing the number of features, which leads to better human interpretations and lower computational costs (Sarker 2021). Anomaly detection identifies observations that deviate significantly from the expected distribution, and is used in applications such as fraud detection and fault monitoring (Goodfellow, Bengio, and Courville 2016, Chapter 5). Semi-supervised learning occupies a middle ground, operating on both labeled and unlabeled data, which is useful in contexts where labeled examples are scarce (Sarker 2021). It is worth noting that the boundary between supervised and unsupervised learning is not always well-defined, and many methods can be applied across both types of problems (Goodfellow, Bengio, and Courville 2016, Chapter 5).

Reinforcement learning differs fundamentally from the other paradigms. Rather than learning from a fixed dataset, an agent interacts with an environment and learns through trial and error, receiving rewards or penalties based on its actions (Bishop 2006) (Sarker 2021). Within reinforcement learning, methods can be broadly divided according to what they learn. In value-based methods, a value function is learned from which the policy is derived either explicitly or implicitly, whereas in policy-based methods the policy is learned directly without the need for a value function (Sewak 2019) (Richard S. Sutton and Barto 2018, Chapter 11). A third class, actor-critic methods, combines both approaches by maintaining a separate policy structure and value function simultaneously (Richard S. Sutton and Barto 2018, Chapter 11).

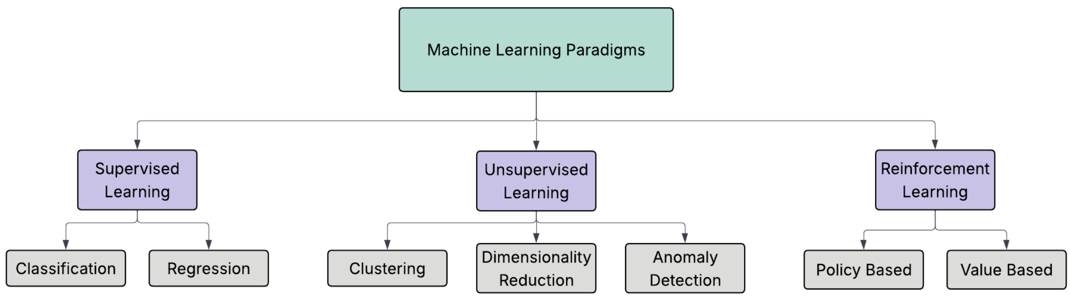


Figure 2.1.1: Overview of machine learning paradigms with common tasks.

2.1.2 Machine Learning Methods

The following sections introduce a selection of common and most established machine learning methods and places them within the paradigms and tasks described above. The aim is not to provide a comprehensive description of each method or to cover underlying mathematical details, but to give an overview of common method types and what fundamentally distinguishes them. The section then gives a brief overview of attributes that are normal to consider when selecting a method for a given problem.

Supervised methods

Tree-based methods learn by recursively partitioning the input space according to feature values. A decision tree splits data into regions by asking a sequence of questions about the input features, assigning each region a predicted output. They can be used for both classification and regression tasks and require little data preparation, such as feature scaling (Geron 2019; Sarker 2021, Chapter 6). Their decisions are intuitive and easy to interpret, as the classification rules they produce can even be applied manually (Geron 2019, Chapter 6). Random forests extend this by training many trees on different subsets of the data and aggregating their predictions, which makes them more robust and generally improves predictive accuracy (Breiman 2001). However, combining many trees makes the model harder to interpret than a single decision tree (Geron 2019, Chapter 6). Gradient boosting methods such as XGBoost build trees sequentially to improve on the predictions of previous trees, and are known for strong generalization performance on large datasets (Tianqi Chen and Guestrin 2016; Sarker 2021).

Linear and polynomial regression predict a continuous output and are among the most widely used methods for regression tasks (Sarker 2021). Regularised variants constrain the model weights to improve generalisation. Ridge regression keeps weights small but never eliminates them entirely, while LASSO regression tends to set some weights exactly to zero, which effectively performs feature selection and produces a sparse model (Tibshirani 1996; Geron 2019). Logistic regression estimates the probability of class membership using a linear combination of input features, and is commonly used for classification tasks, particularly when a simple and interpretable model is preferred (Geron 2019; Sarker 2021, Chapter 4).

Support vector machines find a decision boundary that maximises the margin between classes (Cortes and Vapnik 1995). By applying kernel functions, SVMs can model non-linear decision boundaries and are effective in high-dimensional spaces. They can be used for both classification and regression tasks (Geron 2019; Sarker 2021, Chapter 5).

K-nearest neighbours classifies a new data point based on the majority class among its closest neighbours in the training data, using a distance measure such as Euclidean distance. The method requires no explicit training step but can be computationally expensive at prediction time for large datasets (Cunningham and Delany 2021; Sarker 2021).

Unsupervised methods

Clustering algorithms group data points based on similarity without the use of labels. K-means partitions data into a fixed number of clusters by assigning each instance to the nearest centroid and updating the centroids iteratively until convergence (Geron 2019, Chapter 9). The number of clusters must be specified in advance, and the algorithm is sensitive to the initial centroid placement (Geron 2019, Chapter 9). DBSCAN takes a different approach by defining clusters as dense regions in the data space, which allows it to find clusters of arbitrary shape and identify outliers as noise without requiring the number of clusters to be specified in advance (Ester et al. 1996; Sarker 2021).

For dimensionality reduction, principal component analysis (PCA) identifies the axes of greatest variance in the data and projects it onto a lower-dimensional space defined by these axes, called principal components (Hotelling 1932; Geron 2019). This reduces computational cost and can aid visualisation while preserving as much of the original variance as possible (Geron 2019; Sarker 2021, Chapter 8).

Anomaly detection methods identify observations that deviate significantly from the expected distribution. Isolation Forest isolates anomalies by randomly partitioning the data, exploiting the fact that anomalies are few and structurally different from normal observations (Liu, Ting, and Zhou n.d.). One-class SVM, a variant of the support vector machine, learns a boundary around normal data and flags observations outside this boundary as anomalies (Geron 2019, Chapter 9).

Reinforcement learning methods

Within reinforcement learning, methods differ primarily in what they learn. Value-based methods such as Q-learning estimate the expected return for each action in a given state and derive a policy from these estimates (Watkins and Dayan 1992). Policy-based methods such as policy gradient algorithms optimise the policy directly, which makes them better suited for problems with continuous or high-dimensional action spaces (Richard S Sutton et al. 1999). Actor-critic methods combine both approaches by maintaining a separate policy and value function simultaneously (Zhang and Yu 2020).

Figure 2.1.2 (on the following page) provides a structured overview of common machine learning methods organized by paradigm and task. It includes the methods described in the sections above, as well as additional commonly used methods not covered in detail here. The overview is based on a combination of taxonomies and categorizations described in the literature, and specifically draws on the works of Sarker (2021), Geron (2019), and Zhang and Yu (2020).

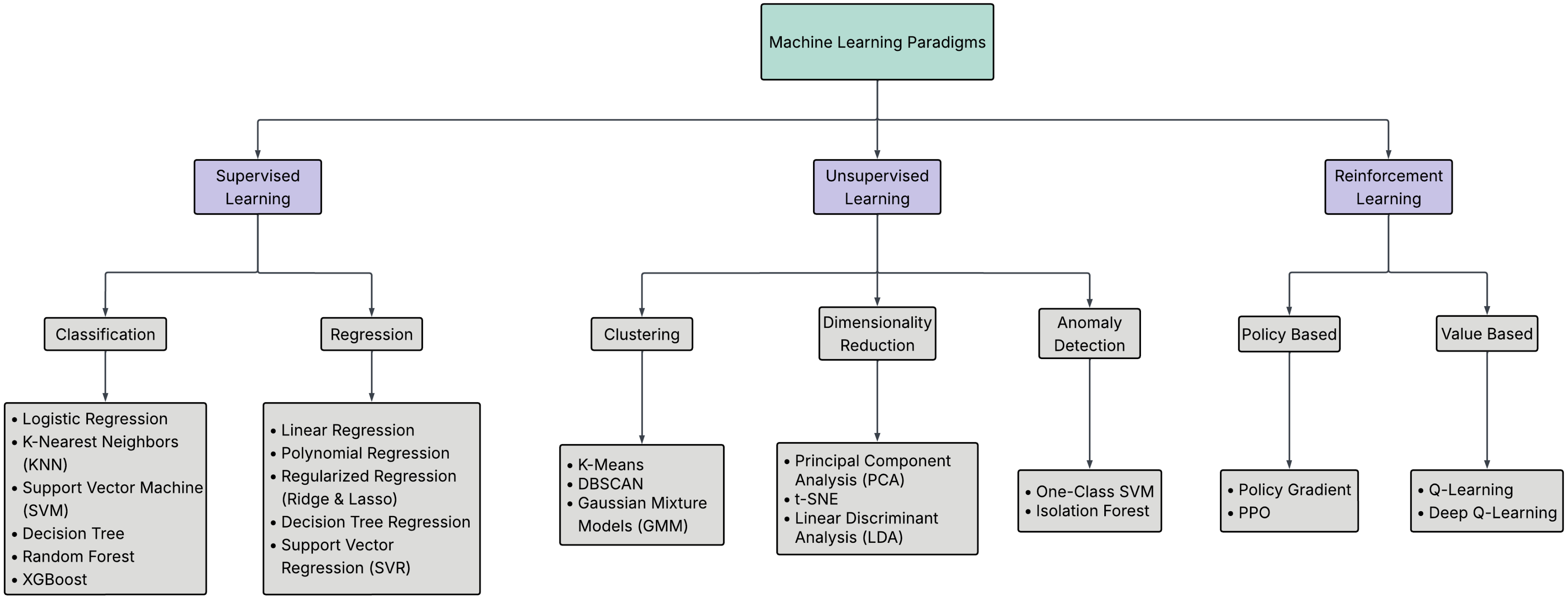


Figure 2.1.2: Overview of common machine learning methods categorized by paradigms and tasks, based on Sarker (2021), Geron (2019), and Zhang and Yu (2020).

2.1.3 Neural Networks and Deep Learning

Neural networks and deep learning are often treated as a separate category in the machine learning literature. The following sections give a brief introduction to some of the most common neural network architectures and their typical use cases.

Neural networks are machine learning models inspired by the structure of the human brain, consisting of processing units organized in input, hidden, and output layers (Shrestha and Mahmood 2019). Deep learning refers to neural networks with many such layers. By stacking multiple layers, the network learns increasingly abstract representations of the input data, where lower layers detect simple patterns and higher layers combine these into more complex concepts (LeCun, Bengio, and Hinton 2015). This makes deep learning particularly well suited for complex, high-dimensional data such as images, audio, and text. (LeCun, Bengio, and Hinton 2015; Alzubaidi et al. 2021). A practical limitation is that deep learning models generally require large amounts of training data to perform well (Alzubaidi et al. 2021).

Feedforward networks and MLP

The simplest deep learning architecture is the multilayer perceptron (MLP), also known as the feedforward neural network. In an MLP, information flows in one direction from input to output through one or more hidden layers, without any feedback connections (Shrestha and Mahmood 2019; Sarker 2021). Each node in one layer connects to each node in the following layer, and the network is trained using backpropagation to adjust the weights (LeCun, Bengio, and Hinton 2015).

Convolutional neural networks

Convolutional neural networks (CNNs) were originally developed for image recognition, where the spatial structure of the input is important (LeCun, Bengio, and Hinton 2015; Lecun et al. 1998). Rather than connecting every neuron to all neurons in the previous layer, CNNs use local connections and shared weights, which reduces the number of parameters and makes the network faster to train (Shrestha and Mahmood 2019). A typical CNN consists of convolutional layers, pooling layers, and fully connected layers. The convolutional layers progressively extract higher-level features from the input, while the pooling layers reduce the spatial size of the representations (Pouyanfar et al. 2018; Sarker 2021). CNNs have achieved strong results in image recognition and computer vision, and are also applied to other data types such as audio and text (LeCun, Bengio, and Hinton 2015; Pouyanfar et al. 2018).

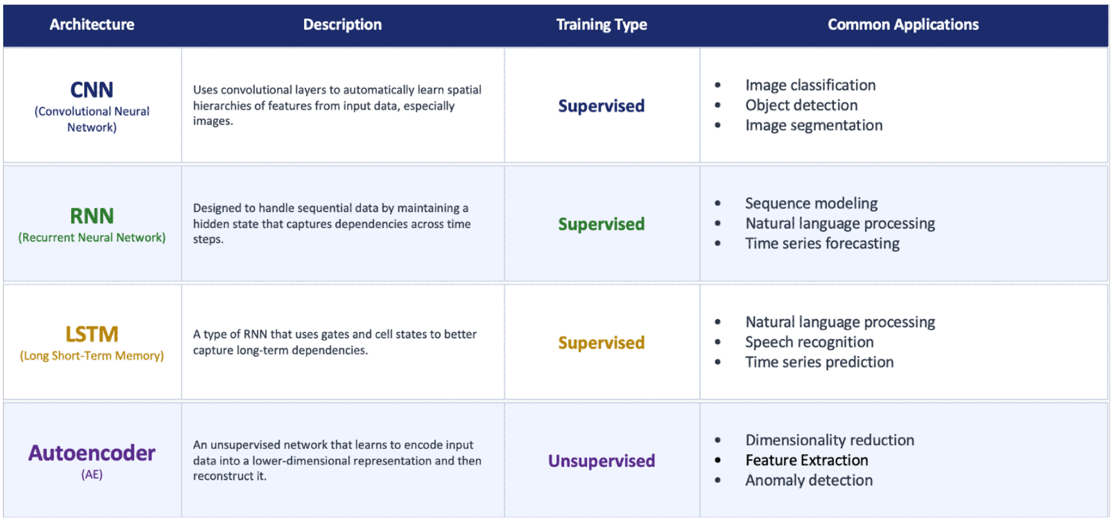
Recurrent neural networks and LSTM

Recurrent neural networks (RNNs) differ from feedforward networks in that the processing units form a cycle, allowing the network to maintain a state based on previous inputs (Shrestha and Mahmood 2019). This makes RNNs well suited for sequential data such as time series, speech, and text (LeCun, Bengio, and Hinton 2015; Pouyanfar et al. 2018). A known limitation of standard RNNs is the vanishing gradient problem, where information about earlier inputs is lost during training as the network processes longer sequences (Pouyanfar et al. 2018; Shrestha and Mahmood 2019).

Long short-term memory networks (LSTMs) address this by introducing memory cells and gating mechanisms that control what information is stored, read, and written (Hochreiter and Schmidhuber 1997; Shrestha and Mahmood 2019). This allows LSTMs to learn from sequences with long-term dependencies, which makes them commonly used for time series analysis, speech recognition, and natural language processing (LeCun, Bengio, and Hinton 2015; Sarker 2021).

Autoencoders

Autoencoders are neural networks trained to compress input data into a lower-dimensional representation and then reconstruct the original input from this representation (Shrestha and Mahmood 2019). Because they learn to reconstruct the input rather than predict a separate target, autoencoders are considered unsupervised models (Shrestha and Mahmood 2019). They are commonly used for dimensionality reduction and anomaly detection, where observations that are difficult to reconstruct may indicate anomalies (Sarker 2021; Shrestha and Mahmood 2019).



Architecture	Description	Training Type	Common Applications
CNN (Convolutional Neural Network)	Uses convolutional layers to automatically learn spatial hierarchies of features from input data, especially images.	**Supervised**	• Image classification • Object detection • Image segmentation
RNN (Recurrent Neural Network)	Designed to handle sequential data by maintaining a hidden state that captures dependencies across time steps.	**Supervised**	• Sequence modeling • Natural language processing • Time series forecasting
LSTM (Long Short-Term Memory)	A type of RNN that uses gates and cell states to better capture long-term dependencies.	**Supervised**	• Natural language processing • Speech recognition • Time series prediction
Autoencoder (AE)	An unsupervised network that learns to encode input data into a lower-dimensional representation and then reconstruct it.	**Unsupervised**	• Dimensionality reduction • Feature Extraction • Anomaly detection

Figure 2.1.3: Summary and comparison of common neural network architectures, based on Sarker (2021), Shrestha and Mahmood (2019), Pouyanfar et al. (2018), and LeCun, Bengio, and Hinton (2015).

Deep reinforcement learning

Deep reinforcement learning (DRL) combines deep learning with reinforcement learning to build agents that can operate in high-dimensional environments. The main motivation for using deep networks in RL is to handle complex, high-dimensional state spaces that cannot be represented by simple tabular or linear methods (Zhang and Yu 2020). A key development in DRL was the deep Q-network (DQN), which uses a neural network to approximate the Q-value function and enabled agents to learn directly from raw inputs such as image pixels

(Zhang and Yu 2020). DRL has demonstrated strong performance in domains such as game playing, robotics, and control, where the state and action spaces are too large for traditional RL methods (Shrestha and Mahmood 2019; Zhang and Yu 2020).

2.1.4 Attributes Relevant for Method Selection (MISSING)

Attributes relevant for method selection include:

- Data type: tabular, time series, image, text
- Data availability: large vs. limited data requirements
- Interpretability: interpretable vs. black-box methods
- Performance preferences: accuracy, training time, computational cost, prediction speed

2.1.5 Approaches to Method Selection

Selecting an appropriate machine learning method is not straightforward. As discussed in Chapter 1.1, the large number of available algorithms, each with distinct strengths and limitations, creates a real challenge for practitioners. This challenge is formally described by the Algorithm Selection Problem (Smith-Miles 2009), which asks which algorithm is likely to perform best for a given problem. The No Free Lunch theorems provide a theoretical basis for this difficulty, showing that no single algorithm performs best across all problems and that good selection requires understanding the characteristics of the problem at hand (Wolpert and Macready 1997).

Several practical tools have been developed to help with method selection. Scikit-learn provides a publicly available decision flowchart that guides users through questions about data size and task type to suggest candidate algorithms (Scikit-learn developers n.d.), and Microsoft Azure offers a similar cheat sheet for its machine learning platform (Microsoft n.d.). While these tools are accessible and widely used, they have clear limitations. The selection criteria used in both tools are mainly accuracy and prediction speed, something that can in practice lead to poor results. (Sala et al. 2018). Factors such as interpretability, data characteristics, and domain context are not considered, and neither do they offer any guidance on how the problem itself should be formulated. (Tingting Chen et al. 2023).

More structured frameworks have been proposed in the academic literature. Sala et al. (2018) present a two-layer selection framework that incorporates algorithm characteristics such as scalability and robustness to outliers, in addition to learning type and response type. Domain-specific frameworks have also been proposed, such as that of Arena et al. (2024), who develop a structured set of decision variables for algorithm selection in predictive maintenance. However, as the authors note, most existing approaches focus narrowly on accuracy and computational requirements, and none provides a tool that connects algorithmic choices to the broader context of the application domain.

A related class of approaches is automated machine learning (AutoML), which seeks to automatically select, compose, and parametrize machine learning models to achieve optimal performance on a given task or dataset (Waring, Lindvall, and Umeton 2020). AutoML systems automate parts of the machine learning pipeline, including data preparation, feature engineering, hyperparameter optimization, and model generation, with the goal of reducing the demand for expert knowledge and making machine learning more accessible to domain experts (He, Zhao, and Chu 2021). Within AutoML, the algorithm selection problem is typically treated as an optimization task, where dataset characteristics are used to predict which algorithm is likely to perform best (Barbudo, Ventura, and Romero 2023).

However, AutoML approaches have notable limitations in the context of decision support for non-experts. Most systems treat the selection and configuration process as a black box, providing little explanation of why a particular configuration was chosen (Barbudo, Ventura, and Romero 2023). This lack of transparency is a significant barrier to adoption, as users who do not understand or trust the output of a system are unlikely to apply it in practice (Waring, Lindvall, and Umeton 2020). Furthermore, AutoML primarily addresses the later stages of the machine learning pipeline, and the phases considered inherently human, such as problem formulation and interpretation of results, have received little attention (Barbudo, Ventura, and Romero 2023). This means that AutoML does not address the earlier challenge of helping practitioners understand which type of problem they have and which general approach is appropriate. This gap motivates the work in this thesis.

Burde siste setning om "this gap" stå her? Eller ta med noe lignende i motivasjon? Eller ingen av delene?

2.2 Machine Learning in Product Development and Production

2.2.1 Applications of Machine Learning

Machine learning is widely applied in industrial product development and production, where it supports tasks such as process optimization, quality prediction, fault diagnosis, predictive maintenance, and anomaly detection (Wuest et al. 2016; Plathottam et al. 2023b). The application of ML in these domains continues to grow, driven by the increasing availability of production data and the improved usability of ML tools (Wuest et al. 2016). Unlike traditional rule-based approaches, ML methods can learn directly from data and adapt to changing conditions, making them well suited to the complex and dynamic nature of manufacturing environments (Wuest et al. 2016). In practice, ML is used both to improve product quality and to optimize production processes, for example through early prediction of manufacturing outcomes, root cause analysis, and condition monitoring of equipment (Weichert et al. 2019). As well as individual processes, ML also supports broader operational tasks such as production planning, scheduling, and supply chain management (Plathottam et al. 2023b; Presciuttini et al. 2024).

2.2.2 Learning Paradigms in Industrial Applications

Across reported applications in manufacturing and production, supervised learning is the dominant paradigm, accounting for the large majority of studies in most application areas (Wuest et al. 2016; Nti et al. 2022b). This reflects the nature of many industrial problems, where labeled data are available through quality inspections, sensor logs, and operator feedback, and where the goal is typically to predict a known outcome such as product quality or equipment failure (Wuest et al. 2016; Presciuttini et al. 2024). Unsupervised learning plays a smaller but important role, particularly in anomaly detection and condition monitoring, where labeled examples of faults are scarce or costly to obtain (Presciuttini et al. 2024). Reinforcement learning is less commonly reported in industrial settings, though it has shown strong results in process control and scheduling problems, where decisions must be made sequentially over time (Panzer and Bender 2022). In practice, the choice of learning paradigm is often shaped by what data is available rather than by the structure of the problem itself (Wuest et al. 2016).

2.2.3 Pre-Master Findings

In a preceding specialization project, a large-scale analysis of machine learning research in product development and production was conducted, classifying over 33,000 article abstracts across product lifecycle phase, application area, learning paradigm, and reported ML methods. The results show that the literature is strongly concentrated in the optimization phase of the product lifecycle, while early phases such as planning and concept development contain comparatively few studies. Supervised learning dominates across all phases and application areas, accounting for approximately 73–80% of the articles in each lifecycle phase, with neural networks, random forests, and support vector machines as the most frequently reported methods. Based on these findings, a roadmap was proposed that structures the literature across lifecycle phase, application area, and learning paradigm to support early-stage method selection (Bergstø 2026).

2.3 Ontologies and Knowledge Representation

2.3.1 Ontology Description

An ontology is an explicit specification of a conceptualization, where a conceptualization is an abstract model of a domain that identifies its relevant concepts and the relationships between them (Gruber 1995). Studer, Benjamins, and Fensel (1998) refine this by describing an ontology as a *formal, explicit specification of a shared conceptualization*. Each term in this definition carries a distinct meaning. *Formal* means the ontology is machine-readable and expressed in a language with precise semantics. *Explicit* means that all concepts and the constraints on their use are clearly defined, rather than left implicit in code or documentation. *Shared* means the ontology captures knowledge that is accepted by a group, rather than reflecting the perspective of a single individual (Studer, Benjamins, and Fensel 1998). In practical terms, an ontology defines a common vocabulary for a domain, including machine-interpretable definitions of its basic concepts and the relations among them (Noy and McGuinness 2001).

An ontology consists of classes, which represent concepts in the domain, properties that describe features of those concepts, and relations that link concepts to one another (Noy and McGuinness 2001). Classes are typically organized in a hierarchy where subclasses inherit the properties of their parent classes. Taken together, these elements allow a domain to be described in a way that both humans and software can interpret and reason over (Gruber 1995).

The design of an ontology involves several practical constraints. Gruber (1995) argues that an ontology should require only the minimal ontological commitment needed for its intended purpose, meaning it should make as few claims about the world as necessary. This principle reflects a trade-off identified by Studer, Benjamins, and Fensel (1998): the more reusable an ontology is, the less directly applicable it tends to be in practice, since a more general representation provides less operational guidance for a specific task. Noy and McGuinness (2001) point to a related constraint on scope. An ontology should not attempt to capture everything that could be said about a domain, but only what is needed for the application at hand. Expanding scope beyond what the application requires adds complexity without benefit, and may introduce inconsistencies that are difficult to resolve. Ontology development is also not a one-time activity. As domain knowledge evolves and new requirements emerge, the ontology must be revised, which means the initial design should anticipate extension and allow the vocabulary to be specialized without requiring changes to existing definitions (Gruber 1995; Noy and McGuinness 2001).

2.3.2 Why ML and Ontology

The machine learning domain has several properties that make ontological representation a natural fit.

ML methods are not a flat collection of independent techniques. They form hierarchical structures where more specific methods are specializations of more general ones. Humm (2026) illustrates this directly: a multilayer perceptron is a specialization of an artificial neural network, and if the latter can be used for classification and regression, the former inherits that applicability without needing it to be stated separately. This kind of reasoning over hierarchical structure is something an ontology supports naturally through inheritance, but is difficult to achieve in a flat list or a conventional database schema.

Beyond hierarchical structure, ML concepts are connected through typed semantic relations. Methods are suited to certain tasks and datatypes, belong to certain learning paradigms, and have different performance characteristics. These associations are not incidental but reflect the structure of the domain itself, and making them explicit is a prerequisite for structured method selection. Keet et al. (2015) motivate their ontology for data mining precisely on these grounds: traditional approaches treated learning algorithms as black boxes, correlating observed performance with data characteristics alone, while an ontology allows the internal characteristics of algorithms, their assumptions, optimization strategies, and decision mechanisms, to be represented explicitly and reasoned over.

2.3.3 Semantic Web Standards

Ontologies both in the ML domain and in general, are commonly implemented using standardised languages from the Semantic Web, a set of specifications developed by the World Wide Web Consortium (W3C) to make information machine-readable and interoperable across systems (World Wide Web Consortium 2026).

The foundation is the Resource Description Framework (RDF), the standard knowledge representation language for the Semantic Web (Gibbins and Shadbolt 2009). RDF represents knowledge as triples, each consisting of a subject, a predicate, and an object, forming a directed graph of binary relationships (Gibbins and Shadbolt 2009). RDF Schema (RDFS) extends RDF with basic constructs for defining classes, properties, and constraints (Gibbins and Shadbolt 2009). Building on RDF, SKOS (Simple Knowledge Organization System) provides a lightweight standard for representing taxonomies and controlled vocabularies, without the formal reasoning capabilities of a full ontology language (Humm 2026).

The Web Ontology Language (OWL) builds on RDF and RDFS and adds richer modelling constructs, including logical operators, quantifiers, and property characteristics such as transitivity (Bechhofer 2009). This gives OWL well-defined semantics that support automated reasoning, making it suitable for ontologies where logical consistency and entailment are required (Bechhofer 2009).

SPARQL (SPARQL Protocol and RDF Query Language) is used to query RDF data. It has syntax similar to SQL and retrieves data by matching triple patterns against stored graphs (Grobe 2009). RDF data is typically stored in triplestores, database systems designed specifically for handling RDF triples (Grobe 2009).

2.3.4 Existing Machine Learning Ontologies

Several ontologies have been developed to represent knowledge in the ML and data mining domain, each with a different scope and purpose. The following is a selection of openly available ontologies identified within the ML domain.

ML-Schema (Publio et al. 2018), developed by the W3C Machine Learning Schema Community Group, is a top-level schema for representing and exchanging metadata about ML algorithms, datasets, models, and experiments. Its primary goal is interoperability. It provides a shared reference vocabulary that other ML ontologies can be mapped to, reducing fragmentation across platforms. ML-Schema defines around 25 abstract classes but contains no populated instances, meaning it describes structure rather than ML content. It is intended as a foundation for more specific ontologies, not as a standalone knowledge base.

OntoDM-core (Panov, Soldatova, and Džeroski 2014) is a formal ontology for core data mining entities such as datasets, tasks, algorithms, and their executions. It is structured across three layers. A specification layer covers abstract definitions, an implementation layer covers algorithm implementations and parameters, and an application layer covers executions and workflows. OntoDM-core is aligned with established upper-level ontologies, which enables its use across different application domains (Panov, Soldatova, and Džeroski 2014). It has been applied in areas such as bioinformatics and drug discovery, and serves as a mid-level ontology for Exposé (Vanschoren et al. 2012), an ontology for ML experiments used in OpenML. With 760 classes and detailed taxonomies of tasks and algorithms, it provides high structural granularity, but contains few populated instances and does not cover named ML methods, metrics, or libraries.

The MEX Vocabulary (Esteves et al. 2015) is a lightweight format for exchanging provenance metadata about ML experiments. It is organised in three layers. MEX-Core handles experiment configuration and execution, MEX-Algorithm covers algorithm class and learning method, and MEX-Performance covers evaluation measures. MEX is designed for basic ML iterations only and does not aim to cover the full data mining domain. Preprocessing semantics, model internals, and named ML methods are outside its scope.

ML Ontology (Humm 2026) is the most comprehensive of the four in terms of ML content. It covers ML areas, tasks, approaches, preprocessing methods, metrics, libraries, AutoML solutions, and configuration parameters, with around 700 individuals and 5000 RDF triples. Unlike the other ontologies, it is not designed for experiment annotation but for integrating different ML tools and frameworks through a shared vocabulary. It supports forward-chaining inference via SPARQL and includes built-in quality assurance checks. Reinforcement learning is listed as a class but contains no instances, and coverage of generative AI methods is limited.

Ontology	Purpose	ML Coverage	Size	Representation	Year
ML Ontology	Shared ML vocabulary for tool integration	Tasks, approaches, metrics, libraries, AutoML solutions, config. items	~10 classes, ~700 individuals, ~5000 triples	RDF/RDFS, SKOS, SPARQL, SHACL	**2026**
ML-Schema	Schema for ML experiment metadata and interoperability	Algorithms, tasks, datasets, models, evaluations.	~25 classes, no instances	OWL/RDF	**2016**
OntoDM-core	Semantic annotation of DM algorithms and processes	DM tasks, algorithms, datasets, generalizations, workflows	760 classes, 221 individuals, 1419 axioms	OWL-DL	**2014**
MEX Vocabulary	Lightweight format for ML experiment metadata	Algorithm class, learning method, hyperparameters, performance measures	~402 classes	RDF, PROV-O	**2015**

Figure 2.3.1: Comparison of existing ML ontologies.

Across these ontologies, a distinction emerges between those more focused on experiment metadata and process structure (ML-Schema, OntoDM-core, MEX) and those focused on ML content and ML method knowledge (ML Ontology). The former are well suited for annotating and reproducing experiments, but provide little support for reasoning about which methods are applicable to a given problem. ML Ontology addresses this gap by representing the relationships between tasks, methods, and metrics in a form that can be queried and reasoned over.

2.3.5 ML Ontology Structure and Content

Among the ontologies reviewed, ML Ontology is examined in more detail here because it provides the broadest coverage of ML concepts and methods, making it the most promising foundation for the system developed in this thesis.

ML Ontology is organised into five modules: classes, instances, inference rules, quality assurance checks, and predefined queries (Humm 2026). Figure 2.3.2 shows this overall structure.

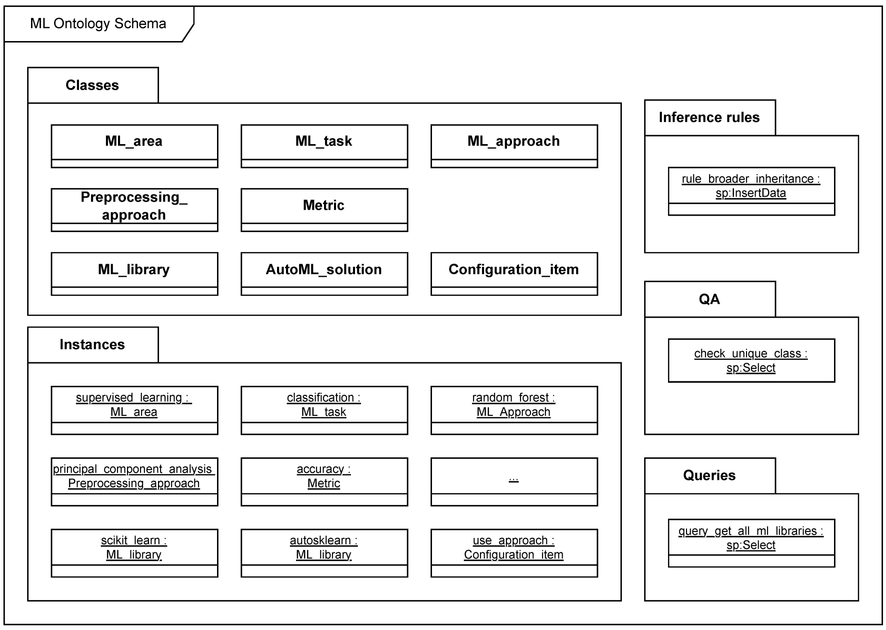


Figure 2.3.2: ML Ontology schema. Source: Humm (2026)

The classes are divided into two groups: ML concepts, which cover the domain vocabulary, and ML implementations, which cover libraries, AutoML solutions, and their configuration options (Humm 2026). Figure 2.3.3 shows the main classes and the relations between them.

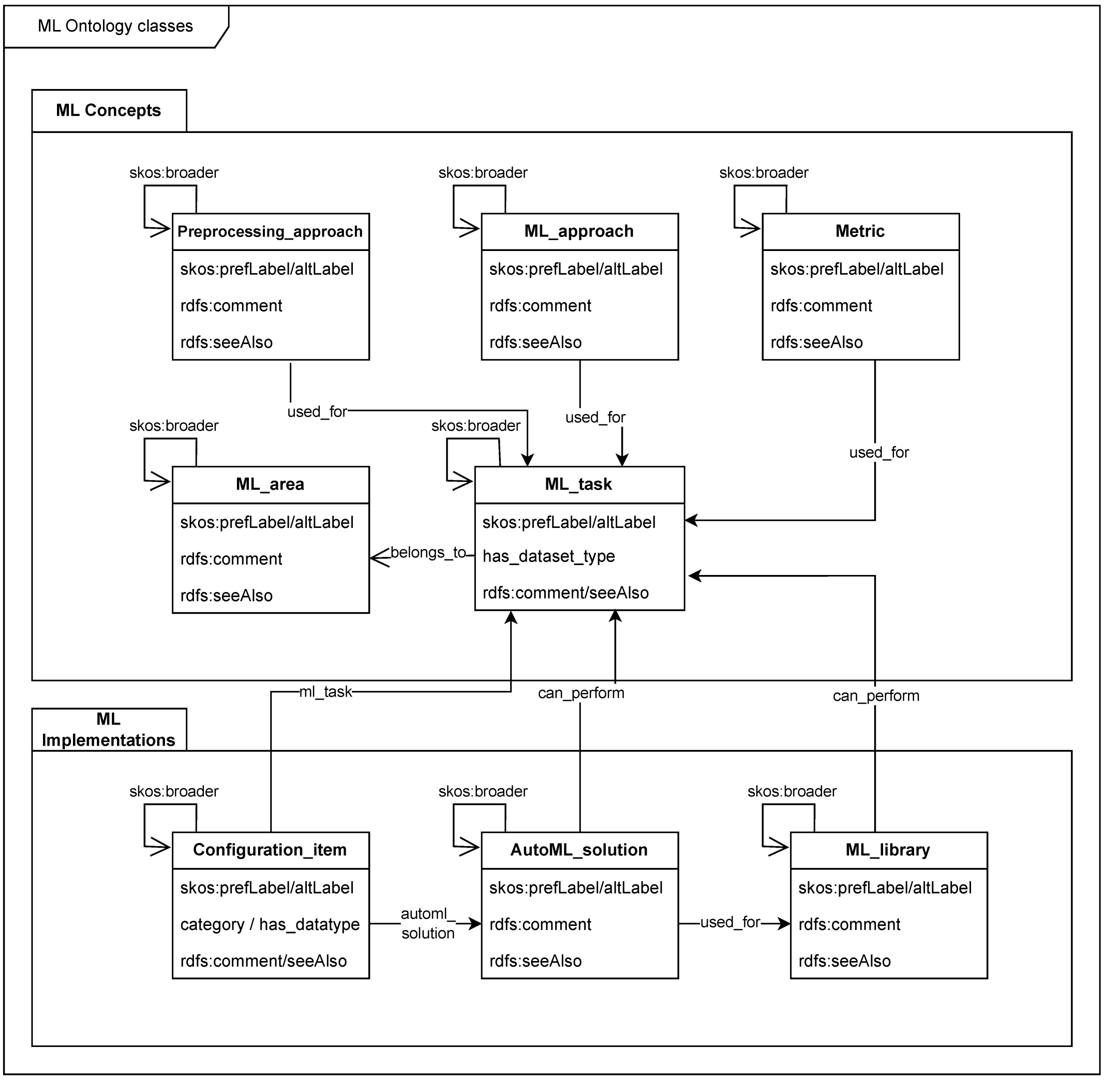


Figure 2.3.3: ML Ontology classes. Source: Humm (2026)

The key relations are `used_for`, which links approaches and metrics to the tasks they apply to, `belongs_to`, which links tasks to learning areas, and `can_perform`, which links libraries and AutoML solutions to tasks (Humm 2026).

ML tasks are organised in hierarchies using `skos:broader`, a SKOS relation that links a concept to a more general one, for example `tabular_regression` to `regression`. A key feature of the ontology is that properties such as `used_for` and `belongs_to` are inherited along these hierarchies through a forward-chaining inference rule (Humm 2026). This means that if a general method is applicable to a task, all its specialisations inherit that applicability without needing it to be stated explicitly. Figure 2.3.4 illustrates the task hierarchy for supervised and unsupervised learning. Reinforcement learning is defined as an ML area in the ontology but does not appear in the hierarchy since no tasks and no methods have been assigned to it (Humm 2026).

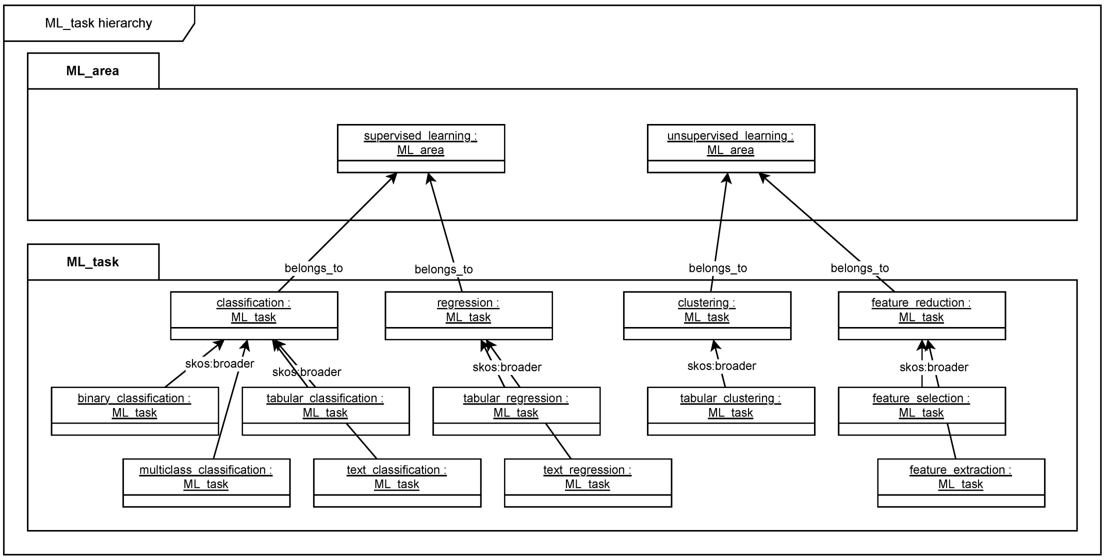


Figure 2.3.4: ML task hierarchy in ML Ontology. Source: Humm (2026)

2.4 Semantic Retrieval and Text Similarity

Ontologies provide structured, explicit representations of domain knowledge, but they depend on manually defined concepts and relations. This makes them well-suited for reasoning over known entities, yet limited in their ability to handle descriptions expressed in natural language or to retrieve knowledge that was not explicitly encoded. Semantic retrieval addresses this gap by operating on the meaning of text rather than on predefined structure. This section introduces the core concepts underlying this approach: how text can be represented as numerical vectors, how similarity between texts can be measured in that representation, and how these techniques support ranking and matching against a corpus of scientific literature.

2.4.1 Text Embeddings and Semantic Similarity

In vector semantics, text is represented as a numerical vector, a point in a high-dimensional space referred to as the embedding space (Jurafsky and Martin 2026, p. 116). The position of a vector in this space reflects the semantic content of the text it represents. A central property of this representation is that semantically similar texts are mapped to nearby points in the embedding space, while unrelated texts are placed further apart. Figure 2.4.1 illustrates this principle for a small set of example words.

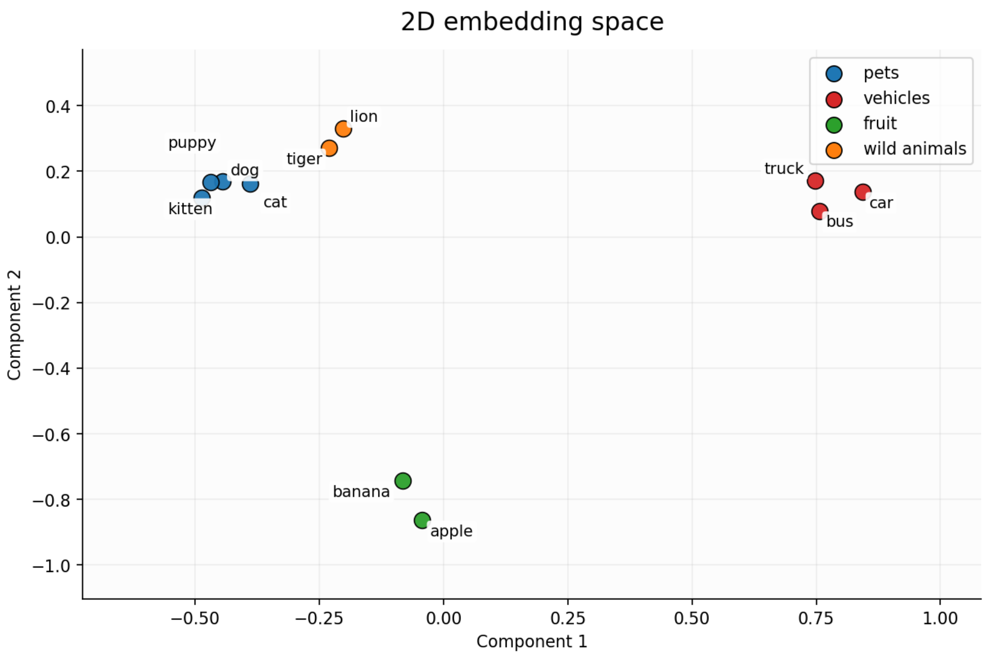


Figure 2.4.1: A two-dimensional projection of example word embeddings. Semantically similar words cluster together in the embedding space, while unrelated words are positioned further apart.

To compare two vectors $\mathbf{u}$ and $\mathbf{v}$, cosine similarity is the most common metric used in natural language processing (Jurafsky and Martin 2026, Chapter 5.4). Rather than measuring absolute distance between two points, it measures the angle between two vectors, normalized for their length:

$$\cos(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|} \tag{2.1}$$

The result ranges from 1, indicating identical direction and maximum similarity, to 0 for orthogonal vectors with no similarity (Jurafsky and Martin 2026, Chapter 5.4). Normalizing for vector length ensures that the similarity score reflects semantic content rather than vector magnitude. In practice, cosine distance, defined as $1 - \cos(\mathbf{u}, \mathbf{v})$, is often used when a distance metric is required, where lower values indicate greater similarity.

2.4.2 Transformer-Based Sentence Embedding Models

Transformer-based language models represent text by processing input tokens through multiple layers of self-attention, producing contextual representations that capture relationships between words across the full input sequence (Vaswani et al. 2023). Unlike earlier embedding approaches that assigned a fixed vector to each word regardless of context, transformer models produce representations that reflect the meaning of a word given its surrounding text.

However, standard transformer models such as BERT are not directly suited for computing sentence-level similarity. As noted by Reimers and Gurevych (2019), comparing two sentences with BERT requires feeding both into the network simultaneously, which becomes computationally infeasible at scale. Finding the most similar pair in a collection of 10,000 sentences would require approximately 50 million inference computations.

Sentence-BERT (SBERT) addresses this by fine-tuning BERT using a siamese network structure, producing fixed-size sentence embeddings that can be compared efficiently using cosine similarity (Reimers and Gurevych 2019). The key property is that semantically similar sentences are placed close together in the resulting embedding space, making SBERT suitable for tasks where embeddings for a large corpus can be precomputed and a new query compared against all of them in milliseconds.

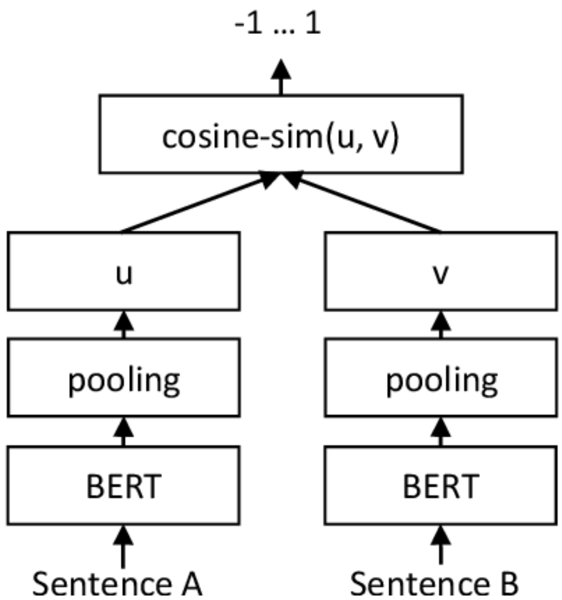


Figure 2.4.2: SBERT architecture at inference time. Each sentence is encoded independently through a shared BERT network, producing vectors **u** and **v** that are compared using cosine similarity. Adapted from Reimers and Gurevych (2019) under CC BY-SA 4.0.

The sentence-transformers library provides pretrained models based on this approach for encoding text into fixed-size vectors (*Sentence-Transformers Documentation* 2026).

2.4.3 Semantic Retrieval

- Retrieval based on semantic meaning rather than exact keyword matching
- A query is embedded and compared against a corpus of pre-embedded documents
- More flexible than keyword search: captures related concepts even when exact terms differ

Information retrieval (IR) is the task of identifying and ranking documents from a collection according to their relevance to a user query (Jurafsky and Martin 2026, Chapter 11.1). Traditional approaches represent both documents and queries as sparse vectors of word counts, weighted by schemes such as tf-idf or BM25, and compute similarity based on shared terms (Jurafsky and Martin 2026, Chapter 11.1). While effective for many tasks, this approach treats documents as bags of words, meaning that word order and meaning are ignored, and only exact term matches contribute to the relevance score.

Dense retrieval addresses this limitation by representing documents and queries as embeddings computed from language models, rather than as word count vectors (Jurafsky and Martin 2026, Chapter 11.1). Because semantically similar texts are mapped to nearby points in the embedding space, a query can retrieve relevant documents even when they do not share exact terms with the query. This makes dense retrieval more flexible than keyword-based approaches for tasks where the user's information need is expressed in natural language rather than precise search terms.

Figure 2.4.3 illustrates the distinction with a constructed example. The same query returns only Document A under keyword-based retrieval, as it is the only document sharing exact terms with the query. Dense retrieval ranks all three documents by semantic similarity, including Documents B and C despite the absence of exact term overlap in these documents.

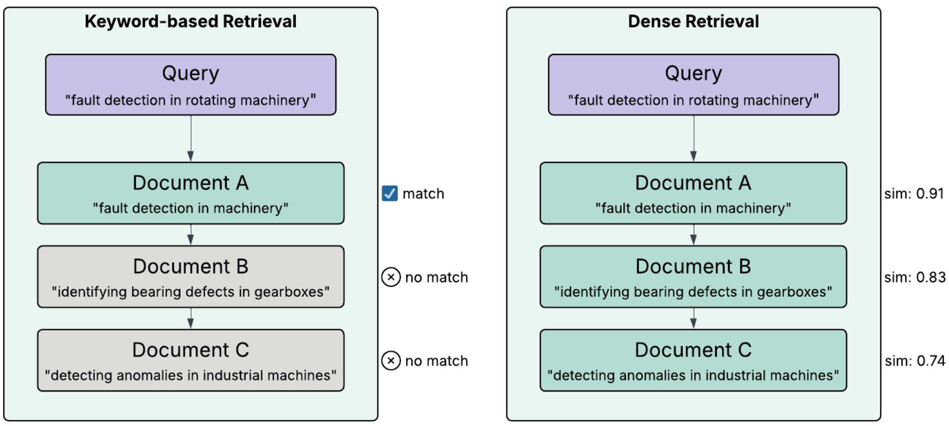


Figure 2.4.3: Keyword-based versus dense retrieval for the same query.

2.4.4 Cross-Encoder Reranking

The sentence embedding approach described in Section **??** is an example of a bi-encoder, where query and document are encoded independently into fixed-size vectors and similarity is measured using cosine similarity (Reimers and Gurevych 2019). Because the representations are computed independently, document embeddings can be precomputed and stored, making bi-encoders efficient for large-scale retrieval (Thakur et al. 2021).

A cross-encoder takes a different approach, concatenating query and document into a single input and processing both jointly, which allows the model to apply attention across both texts simultaneously (Reimers and Gurevych 2019). This produces a direct relevance score rather than a similarity between independent vectors. Cross-encoders are generally more accurate than bi-encoders, but since no independent representations are computed, every query-document pair must be processed from scratch at query time, making cross-encoders impractical for retrieval over large document collections (Thakur et al. 2021).

Figure 2.4.4 illustrates the architectural difference between the two approaches.

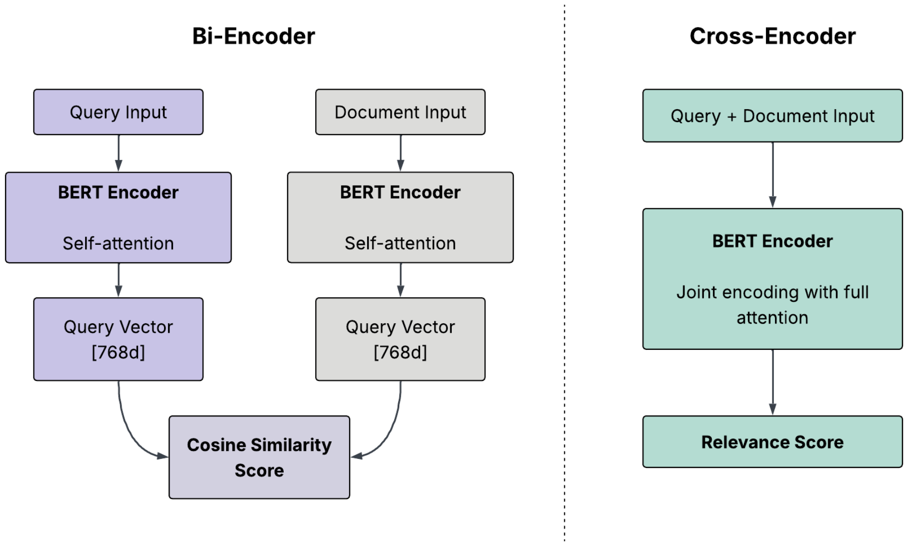


Figure 2.4.4: Bi-encoder and cross-encoder architectures. The bi-encoder encodes query and document independently, producing vectors that are compared using cosine similarity. The cross-encoder concatenates both inputs and processes them jointly, enabling full attention between query and document inputs and producing a direct relevance score.

A common approach to combining the strengths of both architectures is a two-stage retrieval pipeline (Nogueira and Cho 2019). In the first stage, a bi-encoder retrieves a set of candidate documents from the full corpus based on embedding similarity. In the second stage, a cross-encoder reranks these candidates by scoring each document against the query directly, producing a more precise relevance ranking (Nogueira and Cho 2019). This approach preserves the scalability of bi-encoder retrieval while benefiting from the higher accuracy of cross-encoder scoring.

2.4.5 Scientific Literature as a Knowledge Source

Scientific articles document how machine learning methods have been applied across a wide range of problems and contexts. In the domain of machine learning for engineering and manufacturing, a large number of such articles exist, covering diverse algorithms and applications (Sala et al. 2018). This breadth of literature is itself a challenge. Relevant knowledge is spread across journals, conference proceedings, and research communities (Nti et al. 2022a), and the volume of available work makes it difficult for any individual to draw connections across topics and domains (Olivetti et al. 2020). The result is that the literature creates extensive knowledge on the topic, while at the same time making the selection of the right approach difficult (Sala et al. 2018).

Semantic retrieval offers a way to address this challenge by enabling systematic matching between a problem description and relevant articles. Article abstracts are a practical textual basis for this purpose. Text mining of scientific literature has mostly been carried out on abstracts due to their availability, and the fact that abstracts consists of shorter sentences presenting only the most important findings of a study (Westergaard et al. 2018). While full-text articles contain richer information, they also include complex tables, references, and speculative discussion sections, making them less consistent as a basis for comparison across a large corpus (Westergaard et al. 2018). Abstracts thus represent a structured and consistently available representation of each article's core content.

2.5 Related Work and Existing Tools

Few existing tools directly address the combination of method selection, knowledge representation, and implementation support that this thesis investigates. Most related tools either focus on automating a specific part of the ML workflow or provide broad access to AI resources without guiding users through choosing a suitable method for their problem. The following sections review the most relevant examples found.

2.5.1 Ontology-Based Meta AutoML

OMA-ML (Zender and Humm 2022) is a system that combines multiple AutoML solutions through the ML Ontology, described in 2.3.5 The ML Ontology is used to filter configuration options and select appropriate AutoML strategies, making the process more transparent than standard AutoML systems. It also supports a wider range of methods by integrating solutions across multiple ML libraries rather than being tied to a single one. Like other AutoML tools, OMA-ML requires the user to provide a labeled dataset and define the ML task upfront.

2.5.2 Template-Based Code Generation

Traingenerator (Rieke 2021) is a web-based tool that generates template Python code and Jupyter notebooks for ML tasks. Users select a task and framework from a menu and receive a working code template covering common steps such as data loading, model setup, training, and evaluation. The generated code can be downloaded as a Python script or notebook, and optionally opened directly in Google Colab.

The tool assumes the user has already decided on a method and framework before use. At the time of writing, the hosted version of Traingenerator produces a runtime error and does not render output correctly, likely due to an outdated Streamlit API call. The tool appeared to function as intended during the development period of this thesis.

2.5.3 AI-on-Demand Platform

The European AI-on-Demand Platform (AIoDP) and its DeployAI implementation offer a broad marketplace of pre-trained models, reusable AI modules, and domain-specific applications supported by cloud and HPC infrastructure (Troumpoukis et al. n.d.). The platform targets SMEs, researchers, and public sector organizations and provides tools for building, training, and deploying AI solutions at scale. Users can browse and filter available solutions to narrow down options relevant to their domain or use case. The primary focus is on access and deployment of existing solutions rather than on guiding users through method selection for a new problem.

CHAPTER

THREE

METHODOLOGY

CHAPTER

FOUR

IMPLEMENTATION

CHAPTER

FIVE

RESULTS

CHAPTER SIX

DISCUSSION

CHAPTER

SEVEN

CONCLUSIONS

REFERENCES

Wuest, Thorsten et al. (2016). "Machine learning in manufacturing: advantages, challenges, and applications". In: *Production & Manufacturing Research* 4.1. doi: 10.1080/21693277.2016.1192517, pp. 23–45. ISSN: null. DOI: 10.1080/21693277.2016.1192517. URL: https://doi.org/10.1080/21693277.2016.1192517.

Rai, Rahul et al. (2021). "Machine learning in manufacturing and industry 4.0 applications". In: *International Journal of Production Research* 59.16. doi: 10.1080/00207543.2021.1956675, pp. 4773–4778. ISSN: 0020-7543. DOI: 10.1080/00207543.2021.1956675. URL: https://doi.org/10.1080/00207543.2021.1956675.

Nti, Isaac Kofi et al. (2022a). "Applications of artificial intelligence in engineering and manufacturing: a systematic review". In: *Journal of Intelligent Manufacturing* 33.6, pp. 1581–1601. ISSN: 1572-8145. DOI: 10.1007/s10845-021-01771-6. URL: https://doi.org/10.1007/s10845-021-01771-6.

Lee, In and Yong Jae Shin (2020). "Machine learning for enterprises: Applications, algorithm selection, and challenges". In: *Business Horizons* 63.2, pp. 157–170. ISSN: 0007-6813. DOI: https://doi.org/10.1016/j.bushor.2019.10.005. URL: https://www.sciencedirect.com/science/article/pii/S0007681319301521.

Plathottam, Siby Jose et al. (2023a). "A review of artificial intelligence applications in manufacturing operations". In: *Journal of Advanced Manufacturing and Processing* 5.3, e10159. DOI: https://doi.org/10.1002/amp2.10159. URL: https://doi.org/10.1002/amp2.10159.

Ali, Rahman, Sungyoung Lee, and Tae Choong Chung (2017). "Accurate multi-criteria decision making methodology for recommending machine learning algorithm". In: *Expert Systems with Applications* 71, pp. 257–278. ISSN: 0957-4174. DOI: https://doi.org/10.1016/j.eswa.2016.11.034. URL: https://www.sciencedirect.com/science/article/pii/S0957417416306698.

Paleyes, Andrei, Raoul-Gabriel Urma, and Neil D. Lawrence (2022). "Challenges in Deploying Machine Learning: A Survey of Case Studies". In: *ACM Comput. Surv.* 55.6, Article 114. ISSN: 0360-0300. DOI: 10.1145/3533378. URL: https://doi.org/10.1145/3533378.

Sonntag, Sebastian et al. (2023). "A PATTERN LANGUAGE APPROACH TO IDENTIFY APPROPRIATE MACHINE LEARNING ALGORITHMS IN THE CONTEXT OF PRODUCT DEVELOPMENT". In: *Proceedings of the Design*

Society 3, pp. 365–374. DOI: 10.1017/pds.2023.37. URL: https://www.cambridge.org/core/product/44BBE29AA01344BC5F50174953C5BA8B.

Chen, Tingting et al. (2023). *Machine Learning in Manufacturing towards Industry 4.0: From 'For Now' to 'Four-Know'*. Electronic Article. DOI: 10.3390/app13031903.

Sala, Roberto et al. (2018). *How to select the right machine learning algorithm: a feature-based, scope-oriented selection framework*.

Arena, Simone et al. (2024). "A conceptual framework for machine learning algorithm selection for predictive maintenance". In: *Engineering Applications of Artificial Intelligence* 133, p. 108340. ISSN: 0952-1976. DOI: https://doi.org/10.1016/j.engappai.2024.108340. URL: https://www.sciencedirect.com/science/article/pii/S0952197624004986.

Presciuttini, Anna et al. (2024). "Machine learning applications on IoT data in manufacturing operations and their interpretability implications: A systematic literature review". In: *Journal of Manufacturing Systems* 74, pp. 477–486. ISSN: 0278-6125. DOI: https://doi.org/10.1016/j.jmsy.2024.04.012. URL: https://www.sciencedirect.com/science/article/pii/S0278612524000773.

Mitchell, Tom M. (1997). *Machine Learning*. New York: McGraw-Hill.

Bishop, Christopher M. (2006). *Pattern Recognition and Machine Learning*. Information Science and Statistics. New York: Springer. ISBN: 978-0387310732.

Goodfellow, Ian, Yoshua Bengio, and Aaron Courville (2016). *Deep Learning*. http://www.deeplearningbook.org. MIT Press.

Sarker, Iqbal H. (2021). "Machine Learning: Algorithms, Real-World Applications and Research Directions". In: *SN Computer Science* 2.3, p. 160. ISSN: 2661-8907. DOI: 10.1007/s42979-021-00592-x. URL: https://doi.org/10.1007/s42979-021-00592-x.

Sewak, Mohit (2019). "Policy-Based Reinforcement Learning Approaches". In: *Deep Reinforcement Learning: Frontiers of Artificial Intelligence*. Ed. by Mohit Sewak. Singapore: Springer Singapore, pp. 127–140. ISBN: 978-981-13-8285-7. DOI: 10.1007/978-981-13-8285-7_10. URL: https://doi.org/10.1007/978-981-13-8285-7_10.

Sutton, Richard S. and Andrew G. Barto (2018). *Reinforcement Learning: An Introduction*. 2nd. Cambridge, MA: MIT Press.

Geron, Aurelien (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems*. O'Reilly Media, Inc. ISBN: 1492032646.

Breiman, Leo (2001). "Random Forests". In: *Machine Learning* 45.1, pp. 5–32. ISSN: 1573-0565. DOI: 10.1023/A:1010933404324. URL: https://doi.org/10.1023/A:1010933404324.

Chen, Tianqi and Carlos Guestrin (Aug. 2016). "XGBoost: A Scalable Tree Boosting System". In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, pp. 785–794. DOI: 10.1145/2939672.2939785. URL: http://dx.doi.org/10.1145/2939672.2939785.

Tibshirani, Robert (1996). "Regression Shrinkage and Selection Via the Lasso". In: *Journal of the Royal Statistical Society: Series B (Methodological)* 58.1, pp. 267–288. ISSN: 0035-9246. DOI: 10.1111/j.2517-6161.1996.tb02080.x. URL: https://doi.org/10.1111/j.2517-6161.1996.tb02080.x.

Cortes, Corinna and Vladimir Vapnik (1995). "Support-vector networks". In: *Machine Learning* 20.3, pp. 273–297. ISSN: 1573-0565. DOI: 10.1007/BF00994018. URL: https://doi.org/10.1007/BF00994018.

Cunningham, Pádraig and Sarah Jane Delany (2021). "k-Nearest Neighbour Classifiers - A Tutorial". In: *ACM Computing Surveys* 54.6, pp. 1–25. ISSN: 1557-7341. DOI: 10.1145/3459665. URL: http://dx.doi.org/10.1145/3459665.

Ester, Martin et al. (1996). *A density-based algorithm for discovering clusters in large spatial databases with noise*. Conference Paper.

Hotelling, H. (1932). "Analysis of a complex of statistical variables into principal components". In: *Journal of Educational Psychology* 24, pp. 417–441. DOI: 10.1037/h0071325.

Liu, F. T., K. M. Ting, and Z. H. Zhou (n.d.). "Isolation Forest". In: *2008 Eighth IEEE International Conference on Data Mining*, pp. 413–422. ISBN: 2374-8486. DOI: 10.1109/ICDM.2008.17.

Watkins, Christopher J. C. H. and Peter Dayan (1992). "Q-learning". In: *Machine Learning* 8.3, pp. 279–292. ISSN: 1573-0565. DOI: 10.1007/BF00992698. URL: https://doi.org/10.1007/BF00992698.

Sutton, Richard S et al. (1999). "Policy Gradient Methods for Reinforcement Learning with Function Approximation". In: *Advances in Neural Information Processing Systems*. Ed. by S. Solla, T. Leen, and K. Müller. Vol. 12. MIT Press. URL: https://proceedings.neurips.cc/paper_files/paper/1999/file/464d828b85b0bed98e80ade0a5c43b0f-Paper.pdf.

Zhang, Hongming and Tianyang Yu (2020). "Taxonomy of Reinforcement Learning Algorithms". In: *Deep Reinforcement Learning: Fundamentals, Research and Applications*. Ed. by Hao Dong, Zihan Ding, and Shanghang Zhang. Singapore: Springer Singapore, pp. 125–133. ISBN: 978-981-15-4095-0. DOI: 10.1007/978-981-15-4095-0_3. URL: https://doi.org/10.1007/978-981-15-4095-0_3.

Shrestha, Ajay and Ausif Mahmood (2019). "Review of Deep Learning Algorithms and Architectures". In: *IEEE Access* PP, pp. 1–1. DOI: 10.1109/ACCESS.2019.2912200.

LeCun, Yann, Yoshua Bengio, and Geoffrey Hinton (2015). "Deep learning". In: *Nature* 521.7553, pp. 436–444. ISSN: 1476-4687. DOI: 10.1038/nature14539. URL: https://doi.org/10.1038/nature14539.

Alzubaidi, Laith et al. (2021). "Review of deep learning: concepts, CNN architectures, challenges, applications, future directions". In: *Journal of Big Data* 8.1, p. 53. ISSN: 2196-1115. DOI: 10.1186/s40537-021-00444-8. URL: https://doi.org/10.1186/s40537-021-00444-8.

Lecun, Y. et al. (1998). "Gradient-based learning applied to document recognition". In: *Proceedings of the IEEE* 86.11, pp. 2278–2324. ISSN: 1558-2256. DOI: 10.1109/5.726791.

Pouyanfar, Samira et al. (2018). "A Survey on Deep Learning: Algorithms, Techniques, and Applications". In: *ACM Comput. Surv.* 51.5, Article 92. ISSN: 0360-0300. DOI: 10.1145/3234150. URL: https://doi.org/10.1145/3234150.

Hochreiter, Sepp and Jürgen Schmidhuber (1997). "Long Short-Term Memory". In: *Neural Computation* 9, pp. 1735–1780. DOI: 10.1162/neco.1997.9.8.1735.

Smith-Miles, Kate A. (2009). "Cross-Disciplinary Perspectives on Meta-Learning for Algorithm Selection". In: *ACM Computing Surveys* 41.1. DOI: 10.1145/1456650.1456656.

Wolpert, David H. and William G. Macready (1997). "No Free Lunch Theorems for Optimization". In: *IEEE Transactions on Evolutionary Computation* 1.1, pp. 67–82. DOI: 10.1109/4235.585893.

Scikit-learn developers (n.d.). *Choosing the right estimator*. Accessed: May 2026. URL: https://scikit-learn.org/stable/machine_learning_map.html.

Microsoft (n.d.). *Machine Learning Algorithm Cheat Sheet for Azure Machine Learning*. Accessed: May 2026. URL: https://learn.microsoft.com/en-us/azure/machine-learning/algorithm-cheat-sheet.

Waring, Jonathan, Charlotta Lindvall, and Renato Umeton (2020). "Automated machine learning: Review of the state-of-the-art and opportunities for healthcare". In: *Artificial Intelligence in Medicine* 104, p. 101822. ISSN: 0933-3657. DOI: https://doi.org/10.1016/j.artmed.2020.101822. URL: https://www.sciencedirect.com/science/article/pii/S0933365719310437.

He, Xin, Kaiyong Zhao, and Xiaowen Chu (2021). "AutoML: A survey of the state-of-the-art". In: *Knowledge-Based Systems* 212, p. 106622. ISSN: 0950-7051. DOI: https://doi.org/10.1016/j.knosys.2020.106622. URL: https://www.sciencedirect.com/science/article/pii/S0950705120307516.

Barbudo, Rafael, Sebastián Ventura, and José Raúl Romero (2023). "Eight years of AutoML: categorisation, review and trends". In: *Knowledge and Information Systems* 65.12, pp. 5097–5149. ISSN: 0219-3116. DOI: 10.1007/s10115-023-01935-1. URL: https://doi.org/10.1007/s10115-023-01935-1.

Plathottam, Siby Jose et al. (2023b). "A review of artificial intelligence applications in manufacturing operations". In: *Journal of Advanced Manufacturing and Processing* 5.3, e10159. DOI: https://doi.org/10.1002/amp2.10159. URL: https://doi.org/10.1002/amp2.10159.

Weichert, Dorina et al. (2019). "A review of machine learning for the optimization of production processes". In: *The International Journal of Advanced Manufacturing Technology* 104.5, pp. 1889–1902. ISSN: 1433-3015. DOI: 10.1007/s00170-019-03988-5. URL: https://doi.org/10.1007/s00170-019-03988-5.

Nti, Isaac Kofi et al. (2022b). "Applications of artificial intelligence in engineering and manufacturing: a systematic review". In: *Journal of Intelligent Manufacturing* 33.6, pp. 1581–1601. ISSN: 1572-8145. DOI: 10.1007/s10845-021-01771-6. URL: https://doi.org/10.1007/s10845-021-01771-6.

Panzer, Marcel and Benedict Bender (2022). "Deep reinforcement learning in production systems: a systematic literature review". In: *International Journal of Production Research* 60.13. doi: 10.1080/00207543.2021.1973138, pp. 4316–4341. ISSN: 0020-7543. DOI: 10.1080/00207543.2021.1973138. URL: https://doi.org/10.1080/00207543.2021.1973138.

Bergstø, Martin (Jan. 2026). "Supporting Method Selection for Machine Learning in Product Development and Production: A Multi-Dimensional Classification of Machine Learning Research". In: Specialization Project.

Gruber, Thomas R. (1995). "Toward principles for the design of ontologies used for knowledge sharing?" In: *International Journal of Human-Computer Studies* 43.5, pp. 907–928. ISSN: 1071-5819. DOI: https://doi.org/10.1006/ijhc.

1995.1081. URL: https://www.sciencedirect.com/science/article/pii/S1071581985710816.

Studer, Rudi, V. Richard Benjamins, and Dieter Fensel (1998). "Knowledge engineering: Principles and methods". In: *Data & Knowledge Engineering* 25.1, pp. 161–197. ISSN: 0169-023X. DOI: https://doi.org/10.1016/S0169-023X(97)00056-6. URL: https://www.sciencedirect.com/science/article/pii/S0169023X97000566.

Noy, N. and Deborah McGuinness (2001). "Ontology Development 101: A Guide to Creating Your First Ontology". In: *Knowledge Systems Laboratory* 32.

Humm, Bernhard (2026). "An Industry-Ready Machine Learning Ontology". In: *Applied Sciences* 16, p. 843. DOI: 10.3390/app16020843.

Keet, C. Maria et al. (2015). "The Data Mining OPtimization Ontology". In: *Journal of Web Semantics* 32, pp. 43–53. ISSN: 1570-8268. DOI: https://doi.org/10.1016/j.websem.2015.01.001. URL: https://www.sciencedirect.com/science/article/pii/S1570826815000025.

World Wide Web Consortium (2026). *World Wide Web Consortium (W3C)*. URL: https://www.w3.org/ (visited on 05/12/2026).

Gibbins, Nicholas and Nigel Shadbolt (2009). "Resource Description Framework (RDF)". In: ISSN: 978-1-4398-9196-4. DOI: 10.1201/b11499-53.

Bechhofer, Sean (2009). "OWL: Web Ontology Language". In: *Encyclopedia of Database Systems*. Ed. by Ling Liu and M. Tamer ÖZsu. Boston, MA: Springer US, pp. 2008–2009. ISBN: 978-0-387-39940-9. DOI: 10.1007/978-0-387-39940-9_1073. URL: https://doi.org/10.1007/978-0-387-39940-9_1073.

Grobe, Michael (2009). *RDF, Jena, SparQL and the 'Semantic Web'*. Conference Paper. DOI: 10.1145/1629501.1629525. URL: https://doi.org/10.1145/1629501.1629525.

Publio, Gustavo et al. (2018). *ML-Schema: Exposing the Semantics of Machine Learning with Schemas and Ontologies*. DOI: 10.48550/arXiv.1807.05351.

Panov, Pance, Larisa Soldatova, and Sašo Džeroski (2014). "Ontology of core data mining entities". In: *Data Mining and Knowledge Discovery* 28, pp. 1222–1265. DOI: 10.1007/s10618-014-0363-0.

Vanschoren, Joaquin et al. (2012). "Experiment databases. A new way to share, organize and learn from experiments". In: *Machine Learning* 87, pp. 127–158.

Esteves, Diego et al. (2015). "MEX vocabulary: a lightweight interchange format for machine learning experiments". In: *Proceedings of the 11th International Conference on Semantic Systems*. ACM, pp. 169–176.

Jurafsky, Daniel and James H. Martin (2026). *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*. 3rd. Online manuscript released January 6, 2026. URL: https://web.stanford.edu/~jurafsky/slp3/.

Vaswani, Ashish et al. (2023). *Attention Is All You Need*. arXiv: 1706.03762 [cs.CL]. URL: https://arxiv.org/abs/1706.03762.

Reimers, Nils and Iryna Gurevych (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*, pp. 3973–3983. DOI: 10.18653/v1/D19-1410.

Sentence-Transformers Documentation (2026). URL: https://www.sbert.net/ (visited on 05/11/2026).

Thakur, Nandan et al. (2021). *Augmented SBERT: Data Augmentation Method for Improving Bi-Encoders for Pairwise Sentence Scoring Tasks*, pp. 296–310. DOI: 10.18653/v1/2021.naacl-main.28.

Nogueira, Rodrigo and Kyunghyun Cho (2019). *Passage Re-ranking with BERT*.

Olivetti, Elsa et al. (2020). "Data-driven materials research enabled by natural language processing and information extraction". In: *Applied Physics Reviews* 7, p. 041317. DOI: 10.1063/5.0021106.

Westergaard, David et al. (2018). "A comprehensive and quantitative comparison of text-mining in 15 million full-text articles versus their corresponding abstracts". In: *PLOS Computational Biology* 14.2, e1005962. DOI: 10.1371/journal.pcbi.1005962. URL: https://doi.org/10.1371/journal.pcbi.1005962.

Zender, Alexander and Bernhard Humm (2022). "Ontology-based Meta AutoML". In: *Integrated Computer-Aided Engineering* 29, pp. 1–16. DOI: 10.3233/ICA-220684.

Rieke, Johannes (2021). *Traingenerator: A Web App to Generate Template Code for Machine Learning*. https://github.com/jrieke/traingenerator. Accessed: 2026-05-12.

Troumpoukis, Antonis et al. (n.d.). "DeployAI: AI-on-Demand Platform and Marketplace for Industrial Applications within Europe". In: *Artificial Intelligence Applications and Innovations. AIAI 2025 IFIP WG 12.5 International Workshops*. Ed. by Antonios Papaleonidas et al. Springer Nature Switzerland, pp. 115–127. ISBN: 978-3-031-97317-8.

APPENDICES